SMART PARKING SYSTEM FOR SMART CITIES

1. PROJECT OVERVIEW

The Smart Parking System for Smart Cities is an IoT-based intelligent parking management solution developed using Python, MQTT messaging, Node-RED, and a real-time dashboard. The system enables parking slot monitoring, vehicle entry and exit management, automated gate operations, and dashboard-based parking analytics.

The project simulates a smart city parking environment where users can view parking availability, reserve parking spaces, and monitor parking operations in real time. Parking events are transmitted through MQTT, processed by Node-RED workflows, and displayed through a web-based dashboard.

Project Member :

Aryan Vijay Bhalkar

2. OBJECTIVES

- Develop an IoT-based smart parking solution.
- Monitor parking slot availability in real time.
- Automate vehicle entry and exit operations.
- Implement MQTT publish-subscribe communication.
- Create a Node-RED dashboard for monitoring.
- Track vehicle entries and exits dynamically.
- Improve parking space utilization.
- Reduce parking search time and congestion.

3. KEY FEATURES

- Real-time parking slot monitoring.
- Parking slot allocation and release.
- Vehicle entry management.
- Vehicle exit management.
- Automated gate control.
- MQTT-based communication.
- Interactive Node-RED dashboard.
- Parking activity logging.
- Vehicle counting system.
- Live parking statistics.

4. USE CASE SCENARIOS

Scenario 1: Smart City Parking

Drivers can locate and use available parking spaces efficiently.

Scenario 2: Commercial Parking Facilities

Shopping malls and office complexes can automate parking management.

Scenario 3: Residential Communities

Residents can monitor parking availability and occupancy.

Scenario 4: Municipal Smart City Projects

Authorities can improve urban mobility and reduce traffic congestion.

5. TECHNICAL APPROACH

Frontend (Dashboard Interface)

• Node-RED Dashboard

• Web-Based User Interface

• Real-Time Parking Visualization

Backend

• Python Programming

IoT Communication

• MQTT Protocol

• Mosquitto MQTT Broker

Workflow Automation

• Node-RED Flow Processing

• Event Routing

Data Storage

• JSON-Based Local Database

• Parking Event Logging

6. IMPLEMENTATION PLAN

Phase Activities Timeline :

Requirement Analysis Define project scope and parking objectives 1 Day

System Design Design parking architecture and workflows 1 Day

MQTT Setup Configure broker and communication channels 1 Day

Backend Development Develop Python parking management system 2 Days

Dashboard Development Create Node-RED dashboard and visualizations 2 Days

Integration & Testing Connect all modules and validate functionality 2 Days

Validation & Optimization Improve reliability and performance 1 Day

7. BENEFITS

- For Drivers

• Reduced parking search time.

• Better parking convenience.

• Faster parking allocation.

• Improved user experience.

- For Parking Operators

• Automated parking management.

• Better utilization of parking spaces.

• Reduced manual intervention.

• Real-time monitoring capabilities.

- For Smart Cities

• Reduced traffic congestion.

• Improved urban mobility.

• Efficient infrastructure utilization.

• Support for smart city initiatives.

- For Developers

• Practical IoT implementation experience.

• MQTT communication knowledge.

• Node-RED workflow development expertise.

• Real-time dashboard development skills.

8. PROJECT FLOW

• Start Smart Parking Application.

• Connect Python application to MQTT Broker.

• Initialize parking slot database.

• Monitor parking slot availability.

• Detect vehicle entry requests.

• Allocate available parking slot.

• Publish parking event through MQTT.

• Process event using Node-RED workflows.

• Update dashboard statistics and slot status.

• Perform automatic gate operations.

• Detect vehicle exit requests.

• Release parking slot.

• Publish exit event through MQTT.

• Update vehicle counters and dashboard analytics.

• Maintain parking logs and statistics.

9. REQUIREMENTS SPECIFICATION

System Requirements

• Windows Operating System / Linux Operating System

• Minimum 4 GB RAM

• Dual-Core Processor or Higher

• Internet Connectivity

• Modern Web Browser

Software Requirements

• Python 3.x

• Node-RED

• Mosquitto MQTT Broker

• Paho MQTT Library

• Visual Studio Code

• NPM (Node Package Manager)

Installation Steps

Install Python Dependencies:

pip install -r requirements.txt

Start MQTT Broker:

mosquitto

Start Node-RED:

node-red

Import flows.json into Node-RED

Deploy Node-RED Flow

Run Smart Parking Application:

python park_system.py

Access Dashboard:

http://localhost:1880/ui

10. RISKS AND MITIGATIONS

Risk Impact Mitigation Strategy

MQTT Broker Failure Communication interruption Automatic reconnection mechanisms

Dashboard Downtime Monitoring unavailable Regular testing and maintenance

Invalid User Input Incorrect parking operations Input validation and verification

Incorrect Slot Allocation Parking inconsistency Status verification before allocation

System Performance Issues Delayed dashboard updates Workflow optimization and testing

Data Synchronization Issues Incorrect dashboard statistics Real-time event validation

11. FUTURE ENHANCEMENTS

- RFID-based vehicle identification.
- Automatic number plate recognition.
- Mobile application integration.
- GPS-based parking navigation.
- Online payment gateway integration.
- Cloud database deployment.
- Artificial Intelligence-based parking prediction.
- Vehicle reservation system.
- Multi-location parking management.
- AWS or Azure cloud deployment.
- Smart city transportation integration.
- Real IoT sensor deployment.

12. CONCLUSION

The “Smart Parking System for Smart Cities” project successfully demonstrates the implementation of IoT communication, workflow automation, dashboard visualization, and intelligent parking management concepts to build an efficient smart parking platform.

The system provides real-time parking slot monitoring, automated vehicle entry and exit management, MQTT-based communication, automated gate control, dashboard analytics, and parking activity tracking. By leveraging modern IoT technologies and event-driven architecture, the project improves parking efficiency, reduces congestion, and enhances the overall parking experience.

This project provides practical experience in IoT systems, MQTT communication, Node-RED workflow development, dashboard visualization, and smart city infrastructure implementation. With future enhancements such as RFID integration, cloud deployment, AI-based analytics, and mobile applications, the platform can evolve into a production-ready smart parking solution suitable for large-scale smart city deployments.